



Graph-Coloring Register Allocation

Advanced Micro-Architectural Interferences and Heuristic Lifetime
Constraints in C++

COMPILER BACKEND TOOL

C++

Problem Statement: The Core Compiler Challenge

NP-Complete Challenge

Register allocation is an NP-Complete problem: mapping an unbounded number of symbolic program variables into a fixed physical hardware register pool K .

Core Goal

Prevent overlapping variable lifetimes from sharing the same physical resource. Any two variables live at the same program point must be assigned distinct registers.

Data Engineering: Variable Web Synthesis

Web Construction

Raw instructions are parsed from `parser.cpp` and `dataStruct.h`. Each instruction carries `+` (definition) or `-` (use) markers. A **greedy merging strategy** groups overlapping intervals into unique **Web** structures.

```
for (auto& existingWeb : mergedWebPoints) {  
    bool overlaps = false;  
    for (int point : currentSet) {  
        if (existingWeb.count(point)) {  
            overlaps = true; break;  
        }  
    }  
    if (overlaps) {  
        existingWeb.insert(  
            currentSet.begin(),  
            currentSet.end());  
        merged = true; break;  
    }  
}
```

Structural Map: The Web Interference Graph

Graph Architecture

Generated by `createGraph.cpp`:

- **Vertices** = Consolidated Live Webs
- **Edges** = Active lifetime overlaps (Interferences)

`WebsInterfere` evaluates overlapping line sets to draw constraints. Optimized to iterate over the smaller program-point set first.

```
bool WebsInterfere(  
    const Web &w1,  
    const Web &w2) {  
    if (w1.progPoints.size()  
        > w2.progPoints.size())  
        return WebsInterfere(w2,w1);  
    for (int point : w1.progPoints)  
        if (w2.progPoints.count(pt)>0)  
            return true;  
    return false;  
}
```


Core Algorithm: Basic Graph Coloring Strategy

Execution flow of `regAlloc::baseAllocation` follows the **Greedy Stack-Based Allocation** optimization phases:

1

Simplification Phase

Push vertices onto an execution stack when structural density (active edges) fits within register capacity K .

2

Selection Phase

Pop vertices off the stack and assign the **lowest free register index**.

Simplification Phase: Stack Construction

```
if (activeEdges < numReg) {  
    colorStack.push(v);  
    removedV.insert(v);  
    working = true;  
    break;  
}
```

Stack Push Logic

When a vertex's active edge count falls below `numReg`, it is safely pushed onto `colorStack` and marked in `removedV`. This reduction continues iteratively until the graph is fully simplified or no further vertices qualify — triggering fallback heuristics.

Fallback Heuristics: Spilling vs. Live-Range Splitting

Two advanced optimization paths from `regAlloc.cpp` handle nodes that fail coloring:

Memory Spilling

High-degree nodes receive flag `-2` and are assigned to `M` (memory stack offsets).

Live-Range Splitting

Long-lived webs are bisected into smaller sub-intervals, lowering structural node degrees across the graph.

```
// divide the points by 2
set<int> partA, partB;
int mid = prevWebCopy.progPoints.size()/2;
int i = 0;
for (int p : prevWebCopy.progPoints) {
    if (i < mid) partA.insert(p);
    else partB.insert(p);
    i++;
}
```

Algorithmic Complexity & Performance Analysis

A breakdown of the time complexity for each major phase in the Graph-Coloring Register Allocation process:

Phase	Complexity	Description
Web Synthesis (Merging)	$O(N \cdot R \cdot L)$	where N is unique variables, R is ranges per variable, and L is set lookup/insertion time.
Graph Construction	$O(V^2 \cdot M \log N)$	driven by the nested evaluation of Web intersections.
Basic Register Allocation	$O(V \cdot E)$	linear scan of vertices and their adjacency constraints.
Spilling Heuristic	$O(S \cdot V \cdot E)$	where S is the number of spill iterations required to reach a colorable state.
Recursive Web Splitting	$O(2^S \cdot V^2)$	exponential growth based on recursion depth S and interference recalculations.
Custom Stack Allocation	$O(V^2 + V \cdot E)$	involves repeated active degree scans and LIFO stack reconstruction.

System Architecture: Interactive & Batch Modes

Control Routing

Control routing logic in `menu.cpp` and `main.cpp`:

- **Interactive Mode:** Dynamic text-driven console state loop for loading files, building graphs, and testing configurations manually.
- **Batch Processing Mode:** Triggered via CLI parameter `-b` for fully automated end-to-end runs.

```
int main(int argc, char* argv[]) {
    ProjData data;
    if (argc >= 5 && string(argv[1]) == "-b")
        runBatchMode(argc,argv,data);
    else
        handleInteractiveMode(data);
    return 0;
}
```

Export Diagnostics

Output reporting engine from `output_file.cpp` generates a clean target report detailing **webs** count and physical **r** register mappings.

```
for (auto v : vertices) {
    Web w = v->getInfo();
    if (w.assignedRegister == -2)
        outFile << "M: web" << w.id << "\n";
    else if (w.assignedRegister >= 0)
        outFile << "r" << w.assignedRegister << ": web" <<
w.id << "\n";
}
```